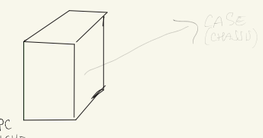
# Introduzione all'hardware

Prima lezione

1. la caratteristica che distingue un computer da un non computer è la riprogrammabilità

## Come è fatto un DESKTOP




Componenti PC
- PERIFERICHE
  - INPUT : dati in ingresso al PC (eg. MOUSE)
  - OUTPUT : (eg MONITOR)
  - I/O (contemporaneamente - schermo touch)
  - MEMORIE DI MASSA - SSD, - HDD } LENTE MA GRANDI e economiche
  - CD - funzionano con memoria ottica - generalmente WRITE-ONE
- MEMORIA CENTRALE:
  - RAM
  - VOLATILE
  - TRANSITORIE (se spengo computer perdo dati)
  - non tutto il contenuto sta su RAM, ma VM (in HDD/SSD)
- CPU : elaborazioni
  - produce molto calore => DISSIPATORI
  - ALU - elaborazioni
  - CU - control UNIT
  - Registri (PC, IR, AX, BX ...) -- memorie piccole e veloci dentro CPU
- MOTHERBOARD
  - collega comunicazione tra CPU, RAM, Perif, ecc
  - contiene CPU, INTERFACCE, ecc
- ALIMENTATORE

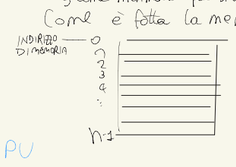
## Macchina di Von Neumann

### Introduzione

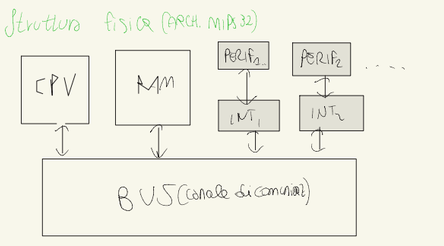
MEMORIA = insieme di BIT (binary digit)
↳ come riferimento però si usano i BYTE (8 bit)

Multipli del BYTE (B)
- KiB = 1024 B = $2^{10}$ B
- MiB = $2^{20}$ B
- GiB = $2^{30}$ B
- TiB = $2^{40}$ B

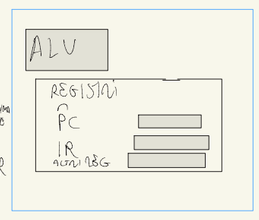
### Come è fatta la memoria




### Struttura fisica (ARCH. MIPS 32)



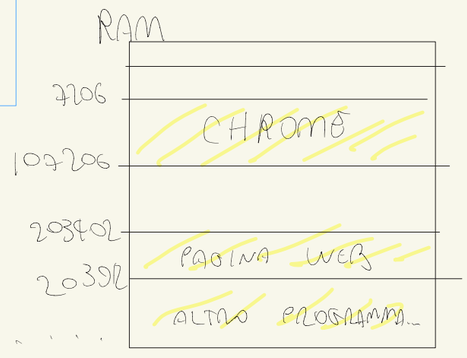

### CPU




PC = PROGRAM COUNTER → CONTIENE ISTRUZIONE DA ESEGUIRE

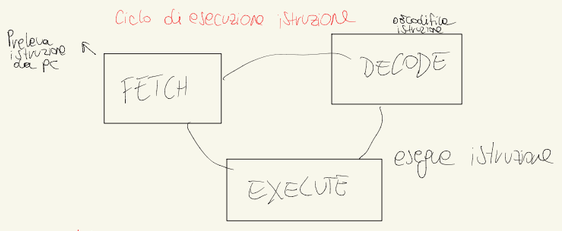
IR = INSTRUCTION REGISTER = ISTRUZIONE ESEGUITA

### RAM




## Cic

### Ciclo di esecuzione istruzione




### Alcune istruzioni Assembly

- il computer esegue codice macchina (0,1)
  ↳ tra quello e il C c'è Assembly

Set istruzioni:

LW = load word
lw $x, Y($0) ⇒ LEGGI IL CONTENUTO DA LOCAZIONE DI MEMORIA Y e lo METTI IN REGISTRO $x

ADD add $x, $y, $z = SCRIVI in x il risultato di y + z

MULT = moltiplica (come ADD ma MULT)

SLT = set if less than
slt $x, $y, $z METTE $x { 1 se $y < $z ; 0 altrimenti }

BEQ = branch if equal - salto condizionato
beq ($x, $y, label)
= INCREMENTA PC di 1W se rispettata condizione, ovvero $x = $y

J = jump salta a istruzione
J x mette PC = x ⇒ esegue x

ADDI = somma un numero (non registro)

SW = store word
sw $x, Y($0) ⇒ METTE IN memoria a locazione Y il contenuto di $x

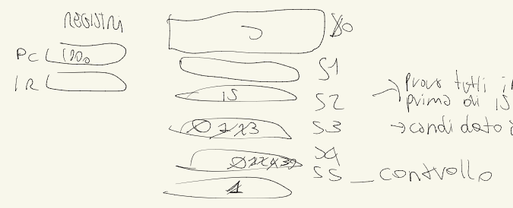
utilizzo BUS perché comunico con CPU

- Queste istruzioni avvengono sulla CPU usando soprattutto la ALU e i REGISTRI
- ECCETTO sw e lw che usano RAM ⇒ BUS per comunicare

Programma in assembly per calcolare la radice quadrata per eccesso

| Indirizzo | Istruzione | |
|---|---|---|
| 1000 | lw | $2, 2000($0) |
| 1004 | ADD | $3, $0, $0 |
| 1008 | ADD | $4, $3, $0 |
| 1012 | MULT | $4, $4, $4 |
| 1016 | SLT | $3, $4, $2 |
| 1020 | BEQ | $3, $0, 18 |
| 1024 | ADDI | $3, $3, 1 |
| 1028 | J | 1008 |
| 1032 | SW | $3, 3004($0) |
| 1036 | ENDP | |

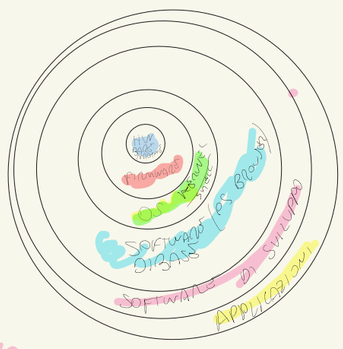
Situaz. partenza memoria
ox 2000 ; 15
ox 3000 ?




→ Prova tutti i numeri prima di 15
→ condi dato alla seconda

## Software

Rappresentazione ONION-LAYERED-SW



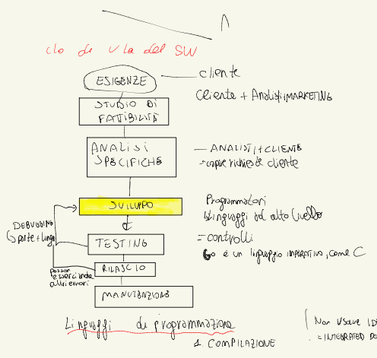
SHELL = interfaccia per utente
KERNEL = il cuore del sistema operativo quello che c'è sotto

Ruolo OS:
- controlla memoria
- controllo CPU - time sharing - esecuz. processi in parallelo
- gestione driver (es per GPU NVIDIA)

## SOFTWARE SVILUPPO

↳ es. COMPILATORE GO

SW BASE es. BROWSER



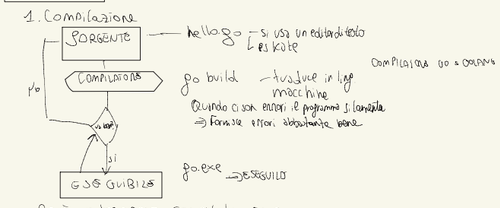

## Linguaggi di programmazione

↳ COMPILAZIONE

Non usare IDE
↳ LINGUAGGIO COMPLETAMENTE ELIMINABILE

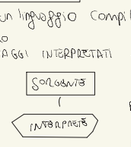
TRADUZIONE

1. Compilazione




Go è un linguaggio compilato, come C

Esistono LINGUAGGI INTERPRETATI — es Python




Programma in Go hellogo

```
package main
import "fmt"
func main(){
  var n int
  fmt.Print("Quante volte vuoi essere salutato/a?")
  fmt.Scan(&n)
  for i=0;i<n;i++{
      fmt.Println("Hello")
  }
}
```

Compilo con go build hello.go
hello.go → hello.exe